

"""

SCQOS — SUBSTRATE-PROOF SINGLE-NODE REFERENCE IMPLEMENTATION

Supreme Full Stack / Nine-Gate Pre-Execution Coherence OS Stack

Python: 3.10+

Runtime dependencies: none

Gates:

1. Time
1. Continuity
1. Alignment
1. Genesis
1. Boundary
1. Reference
1. Causality
1. Consciousness / Substrate
1. Coherence

Every SCQOS module must pass every gate before execution is permitted.
The Coherence Gate receives the real state hashes produced by Gates 1–8.

"""

```
from **future** import annotations
```

```
import json
import math
import hmac
import time
import uuid
import hashlib
import os
import platform
import threading
import warnings
from dataclasses import dataclass, field
from typing import Any, Dict, Optional, Set, List
```

```
#
```

```
—
```

```
# SHARED CONSTANTS
```

#

```
MODULES: List[str] = [  
    "runtime_shell",  
    "policy_registry",  
    "access_compliance",  
    "infrastructure_economy",  
    "io_matrix",  
    "control_surface",  
    "security_layer",  
    "hardware_bridge",  
    "coherence_engine",  
]
```

```
MODULE_OBJECTIVES: Dict[str, str] = {  
    "runtime_shell": "runtime_load",  
    "policy_registry": "enforce_policy",  
    "access_compliance": "verify_access",  
    "infrastructure_economy": "allocate_resource",  
    "io_matrix": "route_io",  
    "control_surface": "operate_control_surface",  
    "security_layer": "secure_boundary",  
    "hardware_bridge": "bridge_hardware",  
    "coherence_engine": "confirm_coherence",  
}
```

```
MODULE_SOURCE_TYPES: Dict[str, str] = {  
    "runtime_shell": "runtime",  
    "policy_registry": "policy",  
    "access_compliance": "security",  
    "infrastructure_economy": "system",  
    "io_matrix": "io",  
    "control_surface": "operator",  
    "security_layer": "security",  
    "hardware_bridge": "hardware",  
    "coherence_engine": "coherence",  
}
```

```
BOUNDARY_SCOPES: Dict[str, str] = {  
    "runtime_shell": "runtime",  
    "policy_registry": "policy",  
    "access_compliance": "security",  
}
```

```
"infrastructure_economy": "resource",  
"io_matrix": "io",  
"control_surface": "control",  
"security_layer": "security",  
"hardware_bridge": "hardware",  
"coherence_engine": "coherence",  
}
```

```
REFERENCE_TARGETS: Dict[str, str] = {  
"runtime_shell": "coherence_engine",  
"policy_registry": "genesis",  
"access_compliance": "security_layer",  
"infrastructure_economy": "policy_registry",  
"io_matrix": "control_surface",  
"control_surface": "access_compliance",  
"security_layer": "boundary",  
"hardware_bridge": "security_layer",  
"coherence_engine": "full_stack",  
}
```

```
MODULE_EFFECT_TYPES: Dict[str, str] = {  
"runtime_shell": "runtime_load",  
"policy_registry": "policy_enforcement",  
"access_compliance": "access_verification",  
"infrastructure_economy": "resource_allocation",  
"io_matrix": "io_routing",  
"control_surface": "control_operation",  
"security_layer": "security_boundary",  
"hardware_bridge": "hardware_bridge",  
"coherence_engine": "coherence_confirmation",  
}
```

```
ALLOWED_OBJECTIVES: Set[str] = {  
"boot",  
"runtime_load",  
"validate_state",  
"route_io",  
"enforce_policy",  
"verify_access",  
"allocate_resource",  
"operate_control_surface",  
"secure_boundary",  
"bridge_hardware",  
"confirm_coherence",  
}
```

}

FORBIDDEN_OBJECTIVES: Set[str] = {

“bypass_gate”,
“skip_validation”,
“disable_audit”,
“forge_signature”,
“replay_state”,
“break_lineage”,
“override_security”,
“unbounded_execution”,
}

ALLOWED_BOUNDARY_DOMAINS: Set[str] = {

“local_node”,
“sandbox”,
“trusted_runtime”,
“scqos_boot”,
}

ALLOWED_SOURCE_TYPES: Set[str] = {

“boot”,
“runtime”,
“policy”,
“operator”,
“system”,
“hardware”,
“security”,
“io”,
“coherence”,
}

REFERENCE_TYPES: Set[str] = {

“time”,
“continuity”,
“alignment”,
“genesis”,
“boundary”,
“module_state”,
“policy_state”,
“security_state”,
“coherence_state”,
}

```
CAUSALITY_EFFECT_TYPES: Set[str] = {
    "boot",
    "runtime_load",
    "state_transition",
    "policy_enforcement",
    "access_verification",
    "resource_allocation",
    "io_routing",
    "control_operation",
    "security_boundary",
    "hardware_bridge",
    "coherence_confirmation",
}
```

```
#
```

```
# SHARED UTILITIES
```

```
#
```

```
def get_secret_key(required: bool = False) -> str:
    key = os.getenv("SCQOS_SECRET_KEY")
    if key:
        return key
    ...
    if required:
        raise RuntimeError(
            "SCQOS_SECRET_KEY environment variable required. "
            "Set SCQOS_SECRET_KEY before production validation."
        )

    warnings.warn(
        "SCQOS_SECRET_KEY not set; using sandbox-only fallback key. "
        "Do not use fallback secrets in production.",
        RuntimeWarning,
        stacklevel=2,
    )
    return "SCQOS_SANDBOX_ONLY_SECRET_DO_NOT_USE_IN_PRODUCTION"
    ...
```

```

def get_substrate_fingerprint_override() -> Optional[str]:
    value = os.getenv("SCQOS_SUBSTRATE_FINGERPRINT_OVERRIDE")
    return value if value and value.strip() else None

def validate_payload(value: Any, path: str = "payload") -> None:
    if value is None or isinstance(value, (bool, int, str)):
        return

    ...

    if isinstance(value, float):
        if not math.isfinite(value):
            raise ValueError(f"{path} contains non-finite float")
        return

    if isinstance(value, (list, tuple)):
        for index, item in enumerate(value):
            validate_payload(item, f"{path}[{index}]")
        return

    if isinstance(value, dict):
        for key, item in value.items():
            if not isinstance(key, str):
                raise TypeError(f"{path} dict keys must be strings")
            validate_payload(item, f"{path}.{key}")
        return

    raise TypeError(f"{path} unsupported type: {type(value).__name__}")
    ...

def canonical_bytes(data: Dict[str, Any], max_bytes: int = 1_048_576) -> bytes:
    validate_payload(data)
    encoded = json.dumps(
        data,
        sort_keys=True,
        separators=(",", ":"),
        ensure_ascii=False,
        allow_nan=False,
    ).encode("utf-8")

    ...

    if len(encoded) > max_bytes:
        raise ValueError(f"state size {len(encoded)} exceeds limit {max_bytes}")

```

```
return encoded
...
```

```
def sha256_hash(encoded: bytes) -> str:
    return hashlib.sha256(encoded).hexdigest()
```

```
def sha3_hash(encoded: bytes) -> str:
    return hashlib.sha3_512(encoded).hexdigest()
```

```
def hmac_sign(key: bytes, encoded: bytes) -> str:
    return hmac.new(key, encoded, digestmod=hashlib.sha3_512).hexdigest()
```

```
def audit_append(path: str, state_hash: str) -> None:
```

```
    try:
```

```
        with open(path, "a", encoding="utf-8") as audit:
```

```
            audit.write(state_hash + "\n")
```

```
        except (OSError, IOError) as error:
```

```
            if not hasattr(audit_append, "memory_log"):
```

```
                audit_append.memory_log = []
```

```
            audit_append.memory_log.append(
```

```
                {
```

```
                    "timestamp": time.time(),
```

```
                    "path": path,
```

```
                    "state_hash": state_hash,
```

```
                    "error": str(error),
```

```
                }
```

```
            )
```

```
            warnings.warn(
```

```
                f"SCQOS audit fallback active for {path}: {error}",
```

```
                RuntimeWarning,
```

```
                stacklevel=2,
```

```
            )
```

```
def compute_substrate_hash(node_id: str = "scqos_local_node", observer_id: str =
    "scqos_observer", substrate_id: str = "scqos_local_substrate") -> str:
```

```
    raw = {
```

```
        "python": platform.python_version(),
```

```
        "implementation": platform.python_implementation(),
```

```
        "system": platform.system(),
```

```
        "machine": platform.machine(),
```

```
        "node_id": node_id,
```

```
        "observer_id": observer_id,
```

```
        "substrate_id": substrate_id,
```

```
    }
```

```
return sha3_hash(canonical_bytes(raw))
```

```
#
```

```
—
```

```
# GATE 1 — TIME
```

```
#
```

```
—
```

```
@dataclass
```

```
class TimeState:
```

```
    module_id: str
```

```
    submitted_at: float = field(default_factory=time.time)
```

```
    submitted_monotonic: float = field(default_factory=time.monotonic)
```

```
    sequence_index: int = 0
```

```
    prior_hash: Optional[str] = None
```

```
    payload: Dict[str, Any] = field(default_factory=dict)
```

```
@dataclass
```

```
class TimeResult:
```

```
    coherent: bool
```

```
    module_id: str
```

```
    drift_ms: float
```

```
    monotonic_drift_ms: float
```

```
    sequence_valid: bool
```

```
    hash_chain_valid: bool
```

```
    payload_valid: bool
```

```
    state_hash: str
```

```
    display_hash: str
```

```
    reason: str
```

```
class TimeGate:
```

```
    def __init__(self, max_drift_ms: float = 500.0):
```

```
        self.max_drift_ms = float(max_drift_ms)
```

```
        self._sequence: Dict[str, int] = {}
```

```
        self._chain: Dict[str, str] = {}
```

```
        self._lock = threading.Lock()
```

```
...
```

```
def _canonical_map(self, state: TimeState) -> Dict[str, Any]:
```

```
    if not isinstance(state.module_id, str) or not state.module_id.strip():
```



```

        raise ValueError("module_id required")
    if state.module_id not in MODULES:
        raise ValueError(f"unknown module {state.module_id}")
    if not math.isfinite(float(state.submitted_at)):
        raise ValueError("submitted_at must be finite")
    if not math.isfinite(float(state.submitted_monotonic)):
        raise ValueError("submitted_monotonic must be finite")
    if not isinstance(state.sequence_index, int) or state.sequence_index < 0:
        raise ValueError("sequence_index must be a non-negative integer")
    if state.prior_hash is not None and not isinstance(state.prior_hash, str):
        raise TypeError("prior_hash must be None or string")
    if not isinstance(state.payload, dict):
        raise TypeError("payload must be dictionary")
    validate_payload(state.payload)

    return {
        "module_id": state.module_id,
        "submitted_at": round(float(state.submitted_at), 6),
        "submitted_monotonic": round(float(state.submitted_monotonic), 6),
        "sequence_index": int(state.sequence_index),
        "prior_hash": state.prior_hash,
        "payload": state.payload,
    }

```

```

def next_state(self, module_id: str, payload: Optional[Dict[str, Any]] = None) -> TimeState:
    with self._lock:
        if module_id not in MODULES:
            raise ValueError(f"unknown module {module_id}")
        return TimeState(
            module_id=module_id,
            submitted_at=time.time(),
            submitted_monotonic=time.monotonic(),
            sequence_index=self._sequence.get(module_id, -1) + 1,
            prior_hash=self._chain.get(module_id),
            payload=payload or {},
        )

```

```

def check(self, state: TimeState) -> TimeResult:
    with self._lock:
        try:
            now = time.time()
            mono_now = time.monotonic()
            drift_ms = abs(now - float(state.submitted_at)) * 1000
            mono_drift_ms = abs(mono_now - float(state.submitted_monotonic)) * 1000

```

```

        encoded = canonical_bytes(self._canonical_map(state))
        current_hash = sha256_hash(encoded)
        drift_ok = drift_ms <= self.max_drift_ms and mono_drift_ms <= self.max_drift_ms
        last_seq = self._sequence.get(state.module_id, -1)
        sequence_ok = state.sequence_index == last_seq + 1
        last_hash = self._chain.get(state.module_id)
        chain_ok = state.prior_hash is None if last_hash is None else state.prior_hash ==
last_hash
        coherent = drift_ok and sequence_ok and chain_ok

    if coherent:
        self._sequence[state.module_id] = state.sequence_index
        self._chain[state.module_id] = current_hash
        reason = "temporal sequence intact"
    else:
        failures = []
        if not drift_ok:
            failures.append(f"drift failed wall={drift_ms:.2f}ms mono={mono_drift_ms:.2f}ms")
        if not sequence_ok:
            failures.append(f"sequence mismatch got={state.sequence_index}
expected={last_seq + 1}")
        if not chain_ok:
            failures.append("hash chain broken")
        reason = " | ".join(failures)

    return TimeResult(
        coherent=coherent,
        module_id=state.module_id,
        drift_ms=drift_ms,
        monotonic_drift_ms=mono_drift_ms,
        sequence_valid=sequence_ok,
        hash_chain_valid=chain_ok,
        payload_valid=True,
        state_hash=current_hash,
        display_hash=current_hash[:16],
        reason=reason,
    )
except Exception as error:
    return TimeResult(
        coherent=False,
        module_id=getattr(state, "module_id", "UNKNOWN"),
        drift_ms=0.0,
        monotonic_drift_ms=0.0,
        sequence_valid=False,

```

```
        hash_chain_valid=False,
        payload_valid=False,
        state_hash="",
        display_hash="",
        reason=str(error),
    )
...

```

```
#
```

```
# GENERIC SIGNED HASH-CHAIN GATE BASE
```

```
#
```

```
@dataclass
class GenericState:
    session_id: str
    node_id: str
    gate_id: str
    module_id: str
    data: Dict[str, Any]
    index: int = 0
    nonce: str = field(default_factory=lambda: uuid.uuid4().hex)
    prior_hash: Optional[str] = None
    submitted_at: float = field(default_factory=time.time)
    payload: Dict[str, Any] = field(default_factory=dict)
    signature: Optional[str] = None

```

```
@dataclass
class SignedChainResult:
    coherent: bool
    module_id: str
    gate_name: str
    index: int
    state_hash: str
    display_hash: str
    reason: str

```

```
class GenericSignedGate:
    gate_name = "generic"

```

```

audit_path = "generic_gate.audit"

...

def __init__(
    self,
    secret_key: str,
    node_id: str,
    max_payload_bytes: int = 1_048_576,
    audit_path: Optional[str] = None,
):
    if not secret_key:
        raise ValueError("secret_key required")
    if not node_id or not str(node_id).strip():
        raise ValueError("node_id required")

    self.secret_key = secret_key.encode("utf-8")
    self.node_id = str(node_id)
    self.max_payload_bytes = int(max_payload_bytes)
    self.audit_path = audit_path or self.audit_path

    self._index: Dict[str, int] = {}
    self._chain: Dict[str, str] = {}
    self._seen_nonces: Set[str] = set()
    self._lock = threading.Lock()

def _key(self, session_id: str, gate_id: str, module_id: str) -> str:
    return f"{session_id}:{gate_id}:{module_id}"

def _validate_data(self, state: GenericState) -> None:
    pass

def _canonical_map(self, state: GenericState) -> Dict[str, Any]:
    required = {
        "session_id": state.session_id,
        "node_id": state.node_id,
        "gate_id": state.gate_id,
        "module_id": state.module_id,
        "nonce": state.nonce,
    }
    for name, value in required.items():
        if not isinstance(value, str) or not value.strip():
            raise ValueError(f"{name} required")

    if state.module_id not in MODULES:

```

```

        raise ValueError(f"unknown module {state.module_id}")
    if state.node_id != self.node_id:
        raise ValueError("node mismatch")
    if not isinstance(state.index, int) or state.index < 0:
        raise ValueError("index must be a non-negative integer")
    if state.prior_hash is not None and not isinstance(state.prior_hash, str):
        raise TypeError("prior_hash must be None or string")
    if not math.isfinite(float(state.submitted_at)):
        raise ValueError("submitted_at must be finite")
    if not isinstance(state.data, dict):
        raise TypeError("data must be dictionary")
    if not isinstance(state.payload, dict):
        raise TypeError("payload must be dictionary")

```

```

validate_payload(state.data)
validate_payload(state.payload)
self._validate_data(state)

```

```

return {
    "session_id": state.session_id,
    "node_id": state.node_id,
    "gate_name": self.gate_name,
    "gate_id": state.gate_id,
    "module_id": state.module_id,
    "data": state.data,
    "index": int(state.index),
    "nonce": state.nonce,
    "prior_hash": state.prior_hash,
    "submitted_at": round(float(state.submitted_at), 6),
    "payload": state.payload,
}

```

```

def _canonical_bytes(self, state: GenericState) -> bytes:
    return canonical_bytes(self._canonical_map(state), self.max_payload_bytes)

```

```

def next_state(
    self,
    session_id: str,
    gate_id: str,
    module_id: str,
    data: Dict[str, Any],
    payload: Optional[Dict[str, Any]] = None,
) -> GenericState:
    with self._lock:

```

```

if module_id not in MODULES:
    raise ValueError(f"unknown module {module_id}")
key = self._key(session_id, gate_id, module_id)
state = GenericState(
    session_id=session_id,
    node_id=self.node_id,
    gate_id=gate_id,
    module_id=module_id,
    data=data,
    index=self._index.get(key, -1) + 1,
    nonce=uuid.uuid4().hex,
    prior_hash=self._chain.get(key),
    submitted_at=time.time(),
    payload=payload or {},
)
state.signature = hmac_sign(self.secret_key, self._canonical_bytes(state))
return state

```

```

def check(self, state: GenericState) -> SignedChainResult:
    with self._lock:
        try:
            if state.nonce in self._seen_nonces:
                raise ValueError("replay detected")
            encoded = self._canonical_bytes(state)
            expected = hmac_sign(self.secret_key, encoded)
            if not hmac.compare_digest(str(state.signature), expected):
                raise ValueError("signature mismatch")
            key = self._key(state.session_id, state.gate_id, state.module_id)
            last_index = self._index.get(key, -1)
            if state.index != last_index + 1:
                raise ValueError(f"{self.gate_name} index mismatch")
            last_hash = self._chain.get(key)
            if last_hash is None and state.prior_hash is not None:
                raise ValueError(f"invalid genesis {self.gate_name}")
            if last_hash is not None and state.prior_hash != last_hash:
                raise ValueError(f"{self.gate_name} lineage broken")

            state_hash = sha3_hash(encoded)
            self._index[key] = state.index
            self._chain[key] = state_hash
            self._seen_nonces.add(state.nonce)
            audit_append(self.audit_path, state_hash)

        return SignedChainResult(

```

```

        coherent=True,
        module_id=state.module_id,
        gate_name=self.gate_name,
        index=state.index,
        state_hash=state_hash,
        display_hash=state_hash[:16],
        reason=f"{self.gate_name} intact",
    )
except Exception as error:
    return SignedChainResult(
        coherent=False,
        module_id=getattr(state, "module_id", "UNKNOWN"),
        gate_name=self.gate_name,
        index=getattr(state, "index", -1),
        state_hash="",
        display_hash="",
        reason=str(error),
    )

def gate(
    self,
    session_id: str,
    gate_id: str,
    module_id: str,
    data: Dict[str, Any],
    payload: Optional[Dict[str, Any]] = None,
) -> SignedChainResult:
    result = self.check(self.next_state(session_id, gate_id, module_id, data, payload))
    if not result.coherent:
        raise RuntimeError(f"{self.gate_name.upper()} GATE BLOCKED {module_id}:
{result.reason}")
    return result
...

#


---


—

# GATES 2–9

#


---


—

```

```

class ContinuityGate(GenericSignedGate):
    gate_name = "continuity"
    audit_path = "continuity_gate.audit"

    ...

    def gate(self, session_id: str, continuity_id: str, module_id: str, payload: Optional[Dict[str, Any]] =
    None) -> SignedChainResult:
        return super().gate(session_id, continuity_id, module_id, {"continuity": "lineage"}, payload)
    ...

```

```

class AlignmentGate(GenericSignedGate):
    gate_name = "alignment"
    audit_path = "alignment_gate.audit"

    ...

    def __init__(
        self,
        secret_key: str,
        node_id: str,
        allowed_objectives: Optional[List[str]] = None,
        forbidden_objectives: Optional[List[str]] = None,
        allowed_boundary_domains: Optional[List[str]] = None,
        substrate_fingerprint_override: Optional[str] = None,
        enforce_substrate: bool = False,
        **kwargs: Any,
    ):
        super().__init__(secret_key, node_id, **kwargs)
        self.allowed_objectives = set(allowed_objectives or ALLOWED_OBJECTIVES)
        self.forbidden_objectives = set(forbidden_objectives or FORBIDDEN_OBJECTIVES)
        self.allowed_boundary_domains = set(allowed_boundary_domains or
        ALLOWED_BOUNDARY_DOMAINS)
        self.substrate_fingerprint = substrate_fingerprint_override or
        compute_substrate_hash(node_id)
        self.enforce_substrate = enforce_substrate

    def _validate_data(self, state: GenericState) -> None:
        objective = state.data.get("declared_objective", "")
        boundary_domain = state.data.get("boundary_domain", "")
        substrate_fingerprint = state.data.get("substrate_fingerprint", "")
        if not state.data.get("intent", "").strip():
            raise ValueError("intent required")
        if objective not in self.allowed_objectives or objective in self.forbidden_objectives:
            raise ValueError(f"objective not allowed: {objective}")
        if not state.data.get("causal_trigger_hash", "").strip():

```



```

        raise ValueError("causal_trigger_hash required")
    if boundary_domain not in self.allowed_boundary_domains:
        raise ValueError(f"boundary domain not allowed: {boundary_domain}")
    if self.enforce_substrate and not hmac.compare_digest(substrate_fingerprint,
self.substrate_fingerprint):
        raise ValueError("substrate fingerprint mismatch")

```

```

def gate(
    self,
    session_id: str,
    alignment_id: str,
    module_id: str,
    intent: str,
    declared_objective: str,
    causal_trigger_hash: str,
    boundary_domain: str = "scqos_boot",
    reference_context: Optional[Dict[str, Any]] = None,
    payload: Optional[Dict[str, Any]] = None,
) -> SignedChainResult:
    data = {
        "intent": intent,
        "declared_objective": declared_objective,
        "causal_trigger_hash": causal_trigger_hash,
        "boundary_domain": boundary_domain,
        "reference_context": reference_context or {},
        "substrate_fingerprint": self.substrate_fingerprint,
    }
    return super().gate(session_id, alignment_id, module_id, data, payload)
...

```

```

class GenesisGate(GenericSignedGate):

```

```

    gate_name = "genesis"
    audit_path = "genesis_gate.audit"

```

```

...

```

```

def __init__(self, secret_key: str, node_id: str, allowed_source_types: Optional[List[str]] = None,
**kwargs: Any):
    super().__init__(secret_key, node_id, **kwargs)
    self.allowed_source_types = set(allowed_source_types or ALLOWED_SOURCE_TYPES)
    self._origin_seen: Dict[str, str] = {}

```

```

def source_hash(self, source: Dict[str, Any]) -> str:
    return sha3_hash(canonical_bytes(source))

```

```

def _validate_data(self, state: GenericState) -> None:
    source_type = state.data.get("source_type", "")
    if not state.data.get("origin_id", "").strip():
        raise ValueError("origin_id required")
    if not state.data.get("creator_id", "").strip():
        raise ValueError("creator_id required")
    if source_type not in self.allowed_source_types:
        raise ValueError(f"source_type not allowed: {source_type}")
    if not state.data.get("source_hash", "").strip():
        raise ValueError("source_hash required")

def check(self, state: GenericState) -> SignedChainResult:
    result = super().check(state)
    if result.coherent:
        origin_key = f"{state.session_id}:{state.gate_id}:{state.data['origin_id']}"
        previous = self._origin_seen.get(origin_key)
        source_hash = state.data["source_hash"]
        if previous is not None and previous != source_hash:
            return SignedChainResult(False, state.module_id, self.gate_name, state.index, "", "",
"origin source hash changed")
        self._origin_seen[origin_key] = source_hash
    return result

def gate(
    self,
    session_id: str,
    genesis_id: str,
    module_id: str,
    origin_id: str,
    creator_id: str,
    source_type: str,
    source_hash: str,
    payload: Optional[Dict[str, Any]] = None,
) -> SignedChainResult:
    data = {
        "origin_id": origin_id,
        "creator_id": creator_id,
        "source_type": source_type,
        "source_hash": source_hash,
    }
    return super().gate(session_id, genesis_id, module_id, data, payload)
...

class BoundaryGate(GenericSignedGate):

```

```

gate_name = "boundary"
audit_path = "boundary_gate.audit"

...

def _validate_data(self, state: GenericState) -> None:
    expected = BOUNDARY_SCOPES[state.module_id]
    if state.data.get("boundary_domain") not in ALLOWED_BOUNDARY_DOMAINS:
        raise ValueError(f"boundary domain not allowed: {state.data.get('boundary_domain')}")
    if state.data.get("boundary_scope") != expected:
        raise ValueError(f"boundary violation: got {state.data.get('boundary_scope')}, expected {expected}")

def gate(self, session_id: str, boundary_id: str, module_id: str, payload: Optional[Dict[str, Any]] =
None) -> SignedChainResult:
    return super().gate(
        session_id,
        boundary_id,
        module_id,
        {
            "boundary_domain": "scqos_boot",
            "boundary_scope": BOUNDARY_SCOPES[module_id],
        },
        payload,
    )
...

class ReferenceGate(GenericSignedGate):
    gate_name = "reference"
    audit_path = "reference_gate.audit"

    ...

    def __init__(self, *args: Any, **kwargs: Any):
        super().__init__(*args, **kwargs)
        self._references: Dict[str, str] = {}

    def _validate_data(self, state: GenericState) -> None:
        target = state.data.get("reference_target", "")
        ref_hash = state.data.get("reference_hash", "")
        ref_type = state.data.get("reference_type", "")
        expected = REFERENCE_TARGETS[state.module_id]
        if target != expected:
            raise ValueError(f"reference target mismatch: got {target}, expected {expected}")
        if not ref_hash.strip():
            raise ValueError("reference_hash required")

```

```

    if ref_type not in REFERENCE_TYPES:
        raise ValueError(f"reference_type not allowed: {ref_type}")

def check(self, state: GenericState) -> SignedChainResult:
    result = super().check(state)
    if result.coherent:
        key = f"{state.session_id}:{state.gate_id}:{state.module_id}:{state.data['reference_target']}"
        previous = self._references.get(key)
        ref_hash = state.data["reference_hash"]
        if previous is not None and previous != ref_hash:
            return SignedChainResult(False, state.module_id, self.gate_name, state.index, "", "",
"reference hash changed")
        self._references[key] = ref_hash
    return result

def gate(
    self,
    session_id: str,
    reference_id: str,
    module_id: str,
    reference_hash: str,
    reference_type: str = "module_state",
    payload: Optional[Dict[str, Any]] = None,
) -> SignedChainResult:
    data = {
        "reference_target": REFERENCE_TARGETS[module_id],
        "reference_hash": reference_hash,
        "reference_type": reference_type,
    }
    return super().gate(session_id, reference_id, module_id, data, payload)
...

class CausalityGate(GenericSignedGate):
    gate_name = "causality"
    audit_path = "causality_gate.audit"
    ...

    def __init__(self, *args: Any, **kwargs: Any):
        super().__init__(*args, **kwargs)
        self._causes: Dict[str, str] = {}
        self._effects: Set[str] = set()

    def _validate_data(self, state: GenericState) -> None:
        effect_type = state.data.get("effect_type", "")

```

```

    if not state.data.get("cause_id", "").strip():
        raise ValueError("cause_id required")
    if not state.data.get("cause_hash", "").strip():
        raise ValueError("cause_hash required")
    if not state.data.get("effect_id", "").strip():
        raise ValueError("effect_id required")
    if effect_type not in CAUSALITY_EFFECT_TYPES:
        raise ValueError(f"effect_type not allowed: {effect_type}")
    expected = MODULE_EFFECT_TYPES[state.module_id]
    if effect_type != expected:
        raise ValueError(f"effect type mismatch: got {effect_type}, expected {expected}")

def check(self, state: GenericState) -> SignedChainResult:
    result = super().check(state)
    if result.coherent:
        cause_key = f"{state.session_id}:{state.gate_id}:{state.data['cause_id']}"
        effect_key = f"{state.session_id}:{state.gate_id}:{state.data['effect_id']}"
        previous = self._causes.get(cause_key)
        cause_hash = state.data["cause_hash"]
        if previous is not None and previous != cause_hash:
            return SignedChainResult(False, state.module_id, self.gate_name, state.index, "", "",
"cause hash changed")
        if effect_key in self._effects:
            return SignedChainResult(False, state.module_id, self.gate_name, state.index, "", "",
"effect already registered")
        self._causes[cause_key] = cause_hash
        self._effects.add(effect_key)
    return result

def gate(
    self,
    session_id: str,
    causality_id: str,
    module_id: str,
    cause_id: str,
    cause_hash: str,
    effect_id: str,
    payload: Optional[Dict[str, Any]] = None,
) -> SignedChainResult:
    data = {
        "cause_id": cause_id,
        "cause_hash": cause_hash,
        "effect_id": effect_id,
        "effect_type": MODULE_EFFECT_TYPES[module_id],

```

```

    }
    return super().gate(session_id, causality_id, module_id, data, payload)
...

```

```

class ConsciousnessGate(GenericSignedGate):

```

```

    gate_name = "consciousness"

```

```

    audit_path = "consciousness_gate.audit"

```

```

...

```

```

def __init__(

```

```

    self,

```

```

    secret_key: str,

```

```

    node_id: str,

```

```

    observer_id: str = "scqos_observer",

```

```

    substrate_id: str = "scqos_local_substrate",

```

```

    substrate_fingerprint_override: Optional[str] = None,

```

```

    enforce_substrate: bool = False,

```

```

    **kwargs: Any,

```

```

):

```

```

    super().__init__(secret_key, node_id, **kwargs)

```

```

    self.observer_id = observer_id

```

```

    self.substrate_id = substrate_id

```

```

    self.substrate_hash = substrate_fingerprint_override or compute_substrate_hash(node_id,
observer_id, substrate_id)

```

```

    self.enforce_substrate = enforce_substrate

```

```

    self._observations: Dict[str, str] = {}

```

```

def observation_hash(self, observation: Dict[str, Any]) -> str:

```

```

    return sha3_hash(canonical_bytes(observation))

```

```

def _validate_data(self, state: GenericState) -> None:

```

```

    if state.data.get("observer_id") != self.observer_id:

```

```

        raise ValueError("observer mismatch")

```

```

    if state.data.get("substrate_id") != self.substrate_id:

```

```

        raise ValueError("substrate_id mismatch")

```

```

    if self.enforce_substrate and not hmac.compare_digest(state.data.get("substrate_hash", ""),
self.substrate_hash):

```

```

        raise ValueError("substrate hash mismatch")

```

```

    if not state.data.get("observation_hash", "").strip():

```

```

        raise ValueError("observation_hash required")

```

```

def check(self, state: GenericState) -> SignedChainResult:

```

```

    result = super().check(state)

```

```

    if result.coherent:

```

```

        key = f"{state.session_id}:{state.gate_id}:{state.data['observer_id']}:{state.module_id}"
        previous = self._observations.get(key)
        observation_hash = state.data["observation_hash"]
        if previous is not None and previous != observation_hash:
            return SignedChainResult(False, state.module_id, self.gate_name, state.index, "", "",
"observation hash changed")
        self._observations[key] = observation_hash
    return result

```

```

def gate(
    self,
    session_id: str,
    consciousness_id: str,
    module_id: str,
    observation_hash: str,
    payload: Optional[Dict[str, Any]] = None,
) -> SignedChainResult:
    data = {
        "observer_id": self.observer_id,
        "substrate_id": self.substrate_id,
        "substrate_hash": self.substrate_hash,
        "observation_hash": observation_hash,
    }
    return super().gate(session_id, consciousness_id, module_id, data, payload)
...

```

```

class CoherenceGate(GenericSignedGate):
    gate_name = "coherence"
    audit_path = "coherence_gate.audit"
...

```

```

def __init__(self, *args: Any, **kwargs: Any):
    super().__init__(*args, **kwargs)
    self._module_coherence: Dict[str, str] = {}

```

```

def combined_hash(self, proof_bundle: Dict[str, Any]) -> str:
    return sha3_hash(canonical_bytes(proof_bundle))

```

```

def _validate_data(self, state: GenericState) -> None:
    for name in [
        "time_hash",
        "continuity_hash",
        "alignment_hash",
        "genesis_hash",

```

```

        "boundary_hash",
        "reference_hash",
        "causality_hash",
        "consciousness_hash",
    ]:
        value = state.data.get(name, "")
        if not isinstance(value, str) or not value.strip():
            raise ValueError(f"{name} required")

def check(self, state: GenericState) -> SignedChainResult:
    result = super().check(state)
    if result.coherent:
        proof_bundle = {
            "module_id": state.module_id,
            "time_hash": state.data["time_hash"],
            "continuity_hash": state.data["continuity_hash"],
            "alignment_hash": state.data["alignment_hash"],
            "genesis_hash": state.data["genesis_hash"],
            "boundary_hash": state.data["boundary_hash"],
            "reference_hash": state.data["reference_hash"],
            "causality_hash": state.data["causality_hash"],
            "consciousness_hash": state.data["consciousness_hash"],
        }
        final_hash = self.combined_hash(proof_bundle)
        module_key = f"{state.session_id}:{state.gate_id}:{state.module_id}:final"
        previous = self._module_coherence.get(module_key)
        if previous is not None and previous != final_hash:
            return SignedChainResult(False, state.module_id, self.gate_name, state.index, "", "",
"coherence proof changed")
        self._module_coherence[module_key] = final_hash
    return result

def gate(
    self,
    session_id: str,
    coherence_id: str,
    module_id: str,
    time_hash: str,
    continuity_hash: str,
    alignment_hash: str,
    genesis_hash: str,
    boundary_hash: str,
    reference_hash: str,
    causality_hash: str,

```



```

        consciousness_hash: str,
        payload: Optional[Dict[str, Any]] = None,
    ) -> SignedChainResult:
        data = {
            "time_hash": time_hash,
            "continuity_hash": continuity_hash,
            "alignment_hash": alignment_hash,
            "genesis_hash": genesis_hash,
            "boundary_hash": boundary_hash,
            "reference_hash": reference_hash,
            "causality_hash": causality_hash,
            "consciousness_hash": consciousness_hash,
        }
        return super().gate(session_id, coherence_id, module_id, data, payload)
    ...

```

#

SUPREME FULL STACK ORCHESTRATOR

#

```

@dataclass
class SupremeCoherenceResult:
    module_id: str
    time_hash: str
    continuity_hash: str
    alignment_hash: str
    genesis_hash: str
    boundary_hash: str
    reference_hash: str
    causality_hash: str
    consciousness_hash: str
    coherence_hash: str
    final_proof: str
    coherent: bool
    reason: str

```

```

class SCQOSSupremeCoherenceStack:
    def **init**(

```

```

self,
secret_key: str,
node_id: str,
session_id: str = "scqos_session",
continuity_id: str = "scqos_primary_continuity",
alignment_id: str = "scqos_primary_alignment",
genesis_id: str = "scqos_primary_genesis",
boundary_id: str = "scqos_primary_boundary",
reference_id: str = "scqos_primary_reference",
causality_id: str = "scqos_primary_causality",
consciousness_id: str = "scqos_primary_consciousness",
coherence_id: str = "scqos_primary_coherence",
creator_id: str = "scqos_architect",
observer_id: str = "scqos_observer",
substrate_id: str = "scqos_local_substrate",
max_drift_ms: float = 500.0,
substrate_fingerprint_override: Optional[str] = None,
enforce_substrate: bool = False,
):
self.secret_key = secret_key
self.node_id = node_id
self.session_id = session_id
self.continuity_id = continuity_id
self.alignment_id = alignment_id
self.genesis_id = genesis_id
self.boundary_id = boundary_id
self.reference_id = reference_id
self.causality_id = causality_id
self.consciousness_id = consciousness_id
self.coherence_id = coherence_id
self.creator_id = creator_id

...

self.time_gate = TimeGate(max_drift_ms=max_drift_ms)
self.continuity_gate = ContinuityGate(secret_key, node_id)
self.alignment_gate = AlignmentGate(
    secret_key,
    node_id,
    substrate_fingerprint_override=substrate_fingerprint_override,
    enforce_substrate=enforce_substrate,
)
self.genesis_gate = GenesisGate(secret_key, node_id)
self.boundary_gate = BoundaryGate(secret_key, node_id)
self.reference_gate = ReferenceGate(secret_key, node_id)

```

```

self.causality_gate = CausalityGate(secret_key, node_id)
self.consciousness_gate = ConsciousnessGate(
    secret_key,
    node_id,
    observer_id=observer_id,
    substrate_id=substrate_id,
    substrate_fingerprint_override=substrate_fingerprint_override,
    enforce_substrate=enforce_substrate,
)
self.coherence_gate = CoherenceGate(secret_key, node_id)

self.results: Dict[str, SupremeCoherenceResult] = {}
self._previous_module_hash = "GENESIS"

def evaluate_module(self, module_id: str) -> SupremeCoherenceResult:
    if module_id not in MODULES:
        raise ValueError(f"unknown module {module_id}")

    try:
        time_state = self.time_gate.next_state(
            module_id,
            {"phase": "supreme_stack", "module": module_id, "gate": "time"},
        )
        time_result = self.time_gate.check(time_state)
        if not time_result.coherent:
            raise RuntimeError(f"TIME FAILED: {time_result.reason}")
        time_hash = time_result.state_hash

        continuity_result = self.continuity_gate.gate(
            self.session_id,
            self.continuity_id,
            module_id,
            {"phase": "supreme_stack", "module": module_id, "gate": "continuity", "time_hash":
time_hash},
        )
        continuity_hash = continuity_result.state_hash

        objective = MODULE_OBJECTIVES[module_id]
        alignment_result = self.alignment_gate.gate(
            self.session_id,
            self.alignment_id,
            module_id,
            intent=f"{module_id} aligns to {objective}",
            declared_objective=objective,

```

```

causal_trigger_hash=continuity_hash,
boundary_domain="scqos_boot",
reference_context={
    "previous_module_hash": self._previous_module_hash,
    "time_hash": time_hash,
    "continuity_hash": continuity_hash,
    "phase": "supreme_stack",
},
payload={"phase": "supreme_stack", "module": module_id, "gate": "alignment"},
)
alignment_hash = alignment_result.state_hash

source_type = MODULE_SOURCE_TYPES[module_id]
source_payload = {
    "phase": "supreme_stack",
    "module": module_id,
    "source_type": source_type,
    "creator": self.creator_id,
    "time_hash": time_hash,
    "continuity_hash": continuity_hash,
    "alignment_hash": alignment_hash,
    "gate": "genesis",
}
source_hash = self.genesis_gate.source_hash(source_payload)
genesis_result = self.genesis_gate.gate(
    self.session_id,
    self.genesis_id,
    module_id,
    origin_id=f"{module_id}_origin",
    creator_id=self.creator_id,
    source_type=source_type,
    source_hash=source_hash,
    payload=source_payload,
)
genesis_hash = genesis_result.state_hash

boundary_result = self.boundary_gate.gate(
    self.session_id,
    self.boundary_id,
    module_id,
    {"phase": "supreme_stack", "module": module_id, "gate": "boundary", "genesis_hash":
genesis_hash},
)
boundary_hash = boundary_result.state_hash

```

```

reference_result = self.reference_gate.gate(
    self.session_id,
    self.reference_id,
    module_id,
    reference_hash=boundary_hash,
    reference_type="module_state",
    payload={"phase": "supreme_stack", "module": module_id, "gate": "reference"},
)
reference_hash = reference_result.state_hash

causality_result = self.causality_gate.gate(
    self.session_id,
    self.causality_id,
    module_id,
    cause_id=f"{module_id}_cause",
    cause_hash=reference_hash,
    effect_id=f"{module_id}_effect",
    payload={"phase": "supreme_stack", "module": module_id, "gate": "causality"},
)
causality_hash = causality_result.state_hash

observation_payload = {
    "phase": "supreme_stack",
    "module": module_id,
    "gate": "consciousness",
    "time_hash": time_hash,
    "continuity_hash": continuity_hash,
    "alignment_hash": alignment_hash,
    "genesis_hash": genesis_hash,
    "boundary_hash": boundary_hash,
    "reference_hash": reference_hash,
    "causality_hash": causality_hash,
}
observation_hash = self.consciousness_gate.observation_hash(observation_payload)
consciousness_result = self.consciousness_gate.gate(
    self.session_id,
    self.consciousness_id,
    module_id,
    observation_hash=observation_hash,
    payload=observation_payload,
)
consciousness_hash = consciousness_result.state_hash

```

```

coherence_result = self.coherence_gate.gate(
    self.session_id,
    self.coherence_id,
    module_id,
    time_hash=time_hash,
    continuity_hash=continuity_hash,
    alignment_hash=alignment_hash,
    genesis_hash=genesis_hash,
    boundary_hash=boundary_hash,
    reference_hash=reference_hash,
    causality_hash=causality_hash,
    consciousness_hash=consciousness_hash,
    payload={"phase": "supreme_stack", "module": module_id, "gate": "coherence"},
)
coherence_hash = coherence_result.state_hash

```

```

result = SupremeCoherenceResult(
    module_id=module_id,
    time_hash=time_hash,
    continuity_hash=continuity_hash,
    alignment_hash=alignment_hash,
    genesis_hash=genesis_hash,
    boundary_hash=boundary_hash,
    reference_hash=reference_hash,
    causality_hash=causality_hash,
    consciousness_hash=consciousness_hash,
    coherence_hash=coherence_hash,
    final_proof=coherence_result.display_hash,
    coherent=True,
    reason="all nine gates coherent",
)
self.results[module_id] = result
self._previous_module_hash = coherence_hash
return result

```

except Exception as error:

```

result = SupremeCoherenceResult(
    module_id=module_id,
    time_hash="",
    continuity_hash="",
    alignment_hash="",
    genesis_hash="",
    boundary_hash="",
    reference_hash="",

```

```

        causality_hash="",
        consciousness_hash="",
        coherence_hash="",
        final_proof="",
        coherent=False,
        reason=str(error),
    )
    self.results[module_id] = result
    return result

```

def boot_all_modules(self) -> bool:

```

    print("=" * 88)
    print(" SCQOS SUPREME FULL STACK COHERENCE ORCHESTRATION")
    print(" Time → Continuity → Alignment → Genesis → Boundary → Reference → Causality
→ Consciousness → Coherence")
    print("=" * 88)

```

all_coherent = True

for module_id in MODULES:

```

    result = self.evaluate_module(module_id)
    status = "[ OK ]" if result.coherent else "[FAIL]"
    proof = result.final_proof if result.final_proof else "NO_PROOF"
    print(f'{status} {module_id:30s} proof={proof}')
    if not result.coherent:
        all_coherent = False
        print(f"    REASON: {result.reason}")

```

```

    print("=" * 88)
    print(" ALL NINE GATES COHERENT — SCQOS CLEARED FOR EXECUTION" if
all_coherent else " FRAGMENTED — EXECUTION BLOCKED")
    print("=" * 88)
    return all_coherent

```

def report(self) -> None:

```

    print("\nSCQOS SUPREME COHERENCE REPORT")
    print("=" * 88)
    print(f'{'MODULE':30s} {'STATUS':12s} {'FINAL PROOF':16s} REASON')
    print("-" * 88)

```

for module_id in MODULES:

```

    result = self.results.get(module_id)
    if result is None:
        print(f'{module_id:30s} {'NOT RUN':12s} {'—':16s} not evaluated")

```

```

        continue
    print(
        f'{module_id:30s} '
        f'{'COHERENT' if result.coherent else 'FRAGMENTED':12s} '
        f'{result.final_proof or '—':16s} '
        f'{result.reason}"
    )

    print("=" * 88)
...

```

```
#
```

```
—
```

```
# ENTRYPOINT
```

```
#
```

```
—
```

```

if **name** == "**main**":
    stack = SCQOSSupremeCoherenceStack(
        secret_key=get_secret_key(required=False),
        node_id=os.getenv("SCQOS_NODE_ID", "node-alpha"),
        substrate_fingerprint_override=get_substrate_fingerprint_override(),
        enforce_substrate=os.getenv("SCQOS_ENFORCE_SUBSTRATE", "0") == "1",
    )
...

```

```

cleared = stack.boot_all_modules()
stack.report()

```

```

if cleared:
    print("\nBooting Supreme Computation Operating System...\n")
    for module in MODULES:
        print(f"[ OK ] Loaded → {module}")
    print("\nSCQOS ONLINE — FULL COHERENCE CONFIRMED")
else:
    print("\nSCQOS BLOCKED — COHERENCE FAILURE")
    raise SystemExit(1)
...

```